

COMSATS University Islamabad,
Sahiwal Campus

SOFTWARE DESIGN DESCRIPTION

(SDD DOCUMENT)

for

NeuraPath

Version 1.0

By

Danish Riasat **CIIT/FA22-BCS-219/SWL**
Usama Abbas **CIIT/FA22-BCS-227/SWL**

Supervisor

Mr. Bilal Shabir Qaisar

Bachelor of Science in Computer Science (2022-2026)

Table of Contents

1 Introduction ---- 8

1.1 Purpose ---- 8

1.2 Scope of the Project ---- 8

1.2.1 Authentication & Security Module ---- 8

1.2.2 Role-Based Profile Management Module ---- 8

1.2.3 AI Resume Enhancer Module ---- 9

1.2.4 AI Career Assistant Module ---- 9

1.2.5 Settings & Configuration Module ---- 9

2 Design Methodology and Software Process Model ---- 10

2.1 Design Methodology ---- 10

2.1.1 Justification for Object-Oriented Design (OOD) ---- 10

2.2 Software Process Model ---- 11

2.2.1 Justification for Agile Process Model ---- 11

3 System Overview ---- 12

3.1 General Description ---- 12

3.2 Context ---- 12

3.3 Design ---- 12

3.4 Background Information ---- 13

4 Architectural Design ---- 14

4.1 Modular Program Structure ---- 14

4.1.1 Presentation Layer (View) ---- 14

4.1.2 Application Logic Layer (ViewModel) ---- 14

4.1.3 Data Access Layer (Repository) ---- 14

4.1.4 Domain Model Layer (Model) ---- 14

4.2 Module Relationships ---- 15

4.3 System Architecture Diagram ---- 15

5 Process Flow / Representation ---- 17

5.1 Major Process: Smart Resume Enhancement ---- 17

5.1.1 Process Description ---- 17

5.1.2 Activity Diagram ---- 17

6 Design Models ---- 19

6.1 Class Diagram ---- 19

6.2 Sequence Diagram ---- 20

6.3 State Transition Diagram ---- 21

6.4 Data Flow Diagrams (DFD) ---- 22

6.4.1 DFD Level 0 (Context Diagram) ---- 22

6.4.2 DFD Level 1 (System Decomposition) ---- 23

6.4.3 DFD Level 2 (AI Services Expansion) ---- 25

7 Data Design ---- 26

7.1 Information Domain Transformation ----26
7.2 Data Organization and Storage ----26
7.2.1 Primary Storage: Cloud Firestore (NoSQL) ----26
7.2.2 Secondary Storage: Cloudinary (Object Storage) ----26
7.2.3 Local Storage: GetStorage ----26
8 Data Dictionary ---- 27
8.1 AuthRepository ----27
8.2 JobSeekerProfileModel ----27
8.3 ProfessionalProfileModel ----28
8.4 ResumeEnhancerViewModel ----28
8.5 StudentProfileModel ----29
8.6 UserModel ----29
9 Algorithm & Implementation ---- 30
9.1 Class: AuthRepository ----30
9.1.1 Function: login(email, password) ----30
9.1.2 Function: signup(email, password) ----30
9.2 Class: ResumeEnhancerViewModel ----31
9.2.1 Function: pickAndAnalyzeResume() ----31
9.3 Class: StudentProfileModel ----32
9.3.1 Function: calculateCompletion() ----32
10 Software Requirements Traceability Matrix ---- 33
11 Human Interface Design ---- 34
11.1 User Perspective Functionality ----34
11.1.1 Onboarding & Authentication: ----34
11.1.2 Dashboard (Home Screen): ----34
11.1.3 AI Resume Enhancer: ----34
11.1.4 Profile Management: ----34
11.2 Screen Images ----35
11.2.1 Launcher Screen (Splash) ----35
11.2.2 Login Screen ----36
11.2.3 Student Home Screen ----37
11.2.4 Smart Resume Analysis ----38
11.2.5 AI Chat Screen ----39
11.2.6 Settings Screen ----40
11.2.7 Profile Management Screen ----41
11.3 Screen Objects and Actions ----42
12 Appendices ---- 43
12.1 Appendix A: Gemini AI Persona Prompt ----43
12.2 Appendix B: Third-Party Libraries ----43
13 References ---- 44

List of Figures

Figure 4.1: Architecture Design 16
Figure 5.1: Activity Diagram 18
Figure 6.1: Class Diagram 19
Figure 6.2: Sequence Diagram 20
Figure 6.3: State Transition Diagram 21
Figure 6.4: DFD Level 0 (Context Diagram) 23
Figure 6.5: DFD Level 1 (System Decomposition) 24
Figure 6.6: DFD Level 2 (AI Services Expansion) 25
Figure 11.1: Launcher Screen (Splash) 35
Figure 11.2: Login Screen 36
Figure 11.3: Student Home Screen 37
Figure 11.4: Resume Analysis 38
Figure 11.5: AI Chat Screen 39
Figure 11.6: Settings Screen 40
Figure 11.7: Profile Management Screen 41

List of Tables

Table 8.1: AuthRepository 27
Table 8.2: JobSeekerProfileModel 27
Table 8.3: ProfessionalProfileModel 28
Table 8.4: ResumeEnhancerViewModel 28
Table 8.5: StudentProfileModel 29
Table 8.6: UserModel 29
Table 10.1: Software Requirements Traceability Matrix 33
Table 11.1: Screen Objects and Actions 42

Revision History

Name	Date	Reason for changes	Version

Application Evaluation History

Comments (by committee) ***include the ones given at scope time both in doc and presentation**	**Action Taken**

Supervised by

Mr. Bilal Shabir Qaisar

Signature: _____

1 Introduction

NeuraPath is an advanced, AI-driven career guidance mobile application designed to revolutionize how individuals navigate their professional journeys. By integrating cutting-edge Artificial Intelligence with a structured career development framework, NeuraPath serves as a personalized digital mentor for Students, Job Seekers, and Professionals.

1.1 Purpose

The primary purpose of **NeuraPath** is to bridge the widening gap between academic preparation and industry requirements. In the current fast-paced job market, candidates often struggle with:

- Identifying the right career path based on their skills.
- Creating resumes that pass Applicant Tracking Systems (ATS).
- Accessing personalized, real-time career advice without high consultancy fees.

NeuraPath addresses these challenges by providing a centralized platform that not only manages professional profiles but also actively enhances employability through AI analysis and intelligent recommendations. The system aims to democratize access to high-quality career guidance, making it accessible to anyone with a smartphone.

1.2 Scope of the Project

The scope of the project, as implemented to date, encompasses a fully functional cross-platform mobile application developed using **Flutter** [1]. The system is architected to support three distinct user roles, each with specialized workflows and data structures.

The following modules and features have been successfully implemented:

1.2.1 Authentication & Security Module

This module ensures secure access and data protection for all users.

- **Sign Up & Login**: Robust email and password authentication using **Firebase Auth** [2].
- **Role Selection**: A mandatory step during registration where users self-identify as a Student, Job Seeker, or Professional. This selection is immutable to ensure data integrity.
- **Account Recovery:** A complete "Forgot Password" flow including email dispatch, OTP verification, and password reset functionality.
- **Session Management**: Persistent login states to enhance user experience.

1.2.2 Role-Based Profile Management Module

A sophisticated data management system that tailors the user experience based on the selected role. Data is stored in role-specific collections in **Cloud Firestore** [2] (student_profiles, job_seeker_profiles, professional_profiles) to ensure scalability and precise data validation.

- **Student Module:**
    - **Academic Tracking**: Fields for University, Degree, Major, and CGPA.
    - **Skill Profiling:** input for technical and soft skills.
    - **Goal Setting:** Tracking of expected graduation and preferred industry domains.
- **Job-Seeker Module:**
    - **Experience Tracking:** Fields for Total Experience and Highest Qualification.
    - **Career Preferences:** Input for Expected Salary Range and Resume Summaries.
- **Professional Module:**
    - **Career History:** Fields for Current Job Title, Company, and Years of Experience.
    - **Expertise:** Detailed skill tracking for senior-level matching.

1.2.3 AI Resume Enhancer Module

A flagship feature powered by the **Gemini Generative AI API** [3] designed to optimize candidate profiles.

- **PDF Parsing:** Integration of read_pdf_text to extract raw text from uploaded resume documents.
- **Intelligent Analysis:** The extracted text is processed by Gemini [3] AI with a specialized prompt to evaluate the resume against modern ATS standards.
- **Actionable Feedback:** The system generates and displays:
    - **ATS Score:** A quantitative metric (0-100).
    - **Strengths & Weaknesses:** Bulleted qualitative analysis.
    - **Keyword Suggestions:** Missing industry-specific terms to improve visibility.

1.2.4 AI Career Assistant Module

An interactive conversational interface that provides real-time guidance.

- **Chat Interface:** A familiar chat UI integrated into the home dashboard.
- **Contextual Advice:** Users can ask questions regarding interview preparation, career shifts, or skill acquisition, receiving instant, AI-generated responses.

1.2.5 Settings & Configuration Module

A centralized hub for user control and application management.

- **Profile Editing**: A dedicated "Edit Profile" interface that allows users to update basic personal information (Name, Phone) and upload profile pictures (stored via **Cloudinary** [4]) without compromising their role assignment.
- **App Info:** Access to privacy policies, help & support, and application version details.

2 Design Methodology and Software Process Model

The Design Methodology and Software Process Model for **NeuraPath** is given below:

2.1 Design Methodology

For the development of **NeuraPath**, we have adopted the **Object-Oriented Design (OOD)** methodology. This choice is driven by the nature of the tools and frameworks used, specifically **Flutter** [1] and the **Dart** programming language, as well as the complexity of the system's requirements.

2.1.1 Justification for Object-Oriented Design (OOD)

I. Framework Alignment (Flutter & Dart):

- **Everything is a Widget:** In Flutter, the UI is built using a composition of Widgets. Each Widget is essentially a Class (Object) that encapsulates its configuration and state. Using OOD allows us to naturally align with the framework's architecture.
- **Dart Language Features:** Dart is a pure object-oriented language supporting classes, mixins, interfaces, and inheritance. Leveraging these features ensures type safety and code robustness.

II. Modularity and Encapsulation:

- NeuraPath handles complex data structures for different user roles (Students, Job Seekers, Professionals). OOD allows us to encapsulate this data into distinct Model classes (e.g., **StudentProfileModel**, **JobSeekerProfileModel**).
- Buisness logic is encapsulated within ViewModel classes (e.g., **AuthViewModel**, **ProfileViewModel**) keeping the codebase organized and preventing "spaghetti code" typical of procedural approaches in large UI applications.

III. Reusability and Inheritance:

- We utilize inheritance to create base classes for shared functionality. For instance, a generic UserProfile class can define common fields like **name** and **email**, while specific role classes extend it to add **cgpa** or **experience**.
- UI components (like custom buttons or input fields) are created as reusable Objects, reducing code duplication across the 20+ screens of the application.

IV. Maintainability:

- OOD promotes separation of concerns. If a bug is found in the "Resume Analysis" logic, we know exactly which class (**ResumeEnhancerViewModel**) to investigate, without affecting the rest of the application.

2.2 Software Process Model

We are following the **Agile Software Development** process model, specifically an **Iterative and Incremental** approach.

2.2.1 Justification for Agile Process Model

I. Iterative Development:

- The project is being built in distinct phases (iterations).
    - Iteration 1: Core Authentication and Role Management.
    - Iteration 2: Profile Systems and Database Integration.
    - Iteration 3: AI Features (Resume Enhancer, Chat).
- This approach allows for the regular delivery of working software components, enabling continuous testing and refinement.

II. Adaptability to Change:

- AI integration (Gemini API [3]) involves experimental tuning of prompts and parameters. The Agile model allows us to refine the AI logic based on testing results without derailing the entire project schedule.
- User feedback (e.g., the need for a dedicated "Edit Profile" screen without role modification) was incorporated mid-development, which would have been difficult in a rigid Waterfall model.

III. Risk Management:

- By tackling high-risk modules (like Firestore [2] Security Rules and API integration) in early sprints, we mitigated the risk of major system failures later in the lifecycle.
- Incremental testing ensures that new features do not break existing functionality (Regression Testing).

IV. User-Centric Focus:

- Agile emphasizes user satisfaction. By constantly reviewing the UI/UX against the requirements (e.g., the "Eco-Professional" theme), we ensure the final product aligns perfectly with the user's needs.

3 System Overview

The detailed overview of our system is given below:

3.1 General Description

NeuraPath is a sophisticated, AI-powered mobile application designed to serve as a comprehensive career development ecosystem. It functions as a centralized platform where users can manage their professional identities, assess their market readiness, and receive personalized guidance.

The system's functionality is categorized into three primary pillars:

I. **Identity Management:** Users create detailed, role-specific profiles that capture their unique academic and professional attributes. This data is not static; it serves as the foundation for all AI-driven insights.

II. **AI-Driven Enhancement:** The system leverages Generative AI (Gemini [3]) to perform complex tasks that typically require human intervention, such as analyzing a resume for ATS compatibility or providing tailored interview advice.

III. **Strategic Guidance:** Through the AI Career Assistant, the system provides real-time, context-aware answers to career queries, effectively democratizing access to high-quality mentorship.

3.2 Context

The project is developed in the context of a rapidly evolving job market where traditional career guidance methods are often inaccessible, expensive, or outdated.

.

- **The Problem:** Students often graduate with skills that do not match industry demands. Job seekers struggle to get past automated Applicant Tracking Systems (ATS) due to poorly formatted resumes. Professionals find it difficult to navigate career pivots without expert advice.

- **The Solution:** NeuraPath addresses these issues by integrating advanced technology directly into the user's pocket. It contextualizes the user's data (e.g., a student's major or a professional's experience) to provide relevant, actionable tools

3.3 Design

The design of **NeuraPath** prioritizes **Usability**, **Scalability**, and **Aesthetics**.

- **User-Centric UI/UX:** The application features a clean, "Eco-Professional" design theme (Sage Green palette: #8BA989, #4E6E58) that reduces cognitive load. Navigation is streamlined through a curved bottom bar, ensuring that core features like the Chat or Profile are never more than one tap away.

- **Modular Architecture:** The system is built on a robust **MVVM (Model-View-ViewModel)** architecture. This ensures that the User Interface (View) is completely decoupled from the Business Logic (ViewModel) and Data Access (Repository). This design allows for independent testing and easy maintenance of individual modules.

- **Cloud-Native Infrastructure:** The system is designed to be serverless, relying on **Firebase** [2] for backend services (Auth, Database) and **Cloudinary** [4] for media management. This ensures high availability and automatic scaling without the need for managing physical servers.

3.4 Background Information

The concept for NeuraPath stems from the observation that while educational institutions provide academic knowledge, they often lack the resources to provide personalized career strategy for every student. Similarly, mid-career professionals often lack a structured support system.

Existing solutions in the market are fragmented:

- **LinkedIn:** Excellent for networking but lacks personalized, AI-driven resume analysis for entry-level candidates.
- **Resume Builders:** Provide templates but rarely offer deep, content-based feedback.
- **Career Counselors:** Highly effective but prohibitively expensive for most students.

NeuraPath was conceived to unify these fragmented services into a single, cohesive mobile application. By utilizing the **Gemini API** [3], the system can mimic the analytical capabilities of a human career counselor at a fraction of the cost and with instant availability.

4 Architectural Design

The Architecture Design of NeuraPath is explained below:

4.1 Modular Program Structure

NeuraPath is architected using the **MVVM (Model-View-ViewModel)** pattern, which promotes a clean separation of concerns and enhances testability. The system is decomposed into four primary layers, each with specific responsibilities:

4.1.1 Presentation Layer (View)

- **Role:** This layer is responsible for the User Interface (UI). It displays data to the user and captures user interactions (taps, text input).
- **Components:** Flutter Widgets located in the lib/ui/ directory (e.g., StudentHomeScreen, ResumeEnhancerScreen).
- **Interaction:** The View is "dumb"; it does not contain business logic. It listens to the ViewModel for state changes and rebuilds itself accordingly.

4.1.2 Application Logic Layer (ViewModel)

- **Role:** This layer acts as the mediator between the View and the Data Layer. It holds the application state (e.g., isLoading, userProfile), validates user input, and processes business rules.
- **Components:** GetX [5] Controllers located in view_models/ directories (e.g., AuthViewModel, ResumeEnhancerViewModel).
- **Interaction:** It exposes reactive variables (using Rx types) that the View subscribes to. It calls methods in the Repository to fetch or save data.

4.1.3 Data Access Layer (Repository)

- **Role:** This layer abstracts the data sources. It provides a clean API for the ViewModel to access data without knowing how or where the data is stored.
- **Components:** Classes in lib/data/repositories/ (e.g., AuthRepository, ProfileRepository).
- **Interaction:** It communicates directly with external services (Firebase, APIs) and maps raw data (JSON/Documents) into Domain Models.

4.1.4 Domain Model Layer (Model)

- **Role**: This layer defines the structure of the data used throughout the app. It ensures type safety and data consistency.
- **Components:** Dart classes in lib/model/ (e.g., UserModel, StudentProfileModel).
- **Interaction:** These are simple data transfer objects (DTOs) passed between layers.

4.2 Module Relationships

The collaboration between these modules follows a strict unidirectional dependency flow:

I. **View -> ViewModel:** The View calls methods on the ViewModel (e.g., viewModel.login()).
II. **ViewModel -> Repository:** The ViewModel requests data from the Repository (e.g., repository.getUser()).
III. **Repository -> External Services:** The Repository makes network requests to Firebase [2] or Gemini [3].
IV. **External Services -> Repository:** Returns raw data.
V. **Repository -> ViewModel:** Returns Model objects.
VI. **ViewModel -> View:** Updates reactive state, causing the View to rebuild.

4.3 System Architecture Diagram

The following diagram illustrates the high-level architecture and the data flow between the subsystems.

The System Architecture Diagram illustrates the high-level structure of the NeuraPath application, following the MVVM (Model-View-ViewModel) design pattern. It visualizes the separation of concerns between the Client Device (Mobile App) and the Cloud Services (Backend).

I. Client Device (NeuraPath Mobile Application):

- **Presentation Layer (View):** Represents the UI screens (e.g.,StudentHomeScreen, ResumeEnhancerScreen) that the user interacts with. It sends user actions to the Logic Layer and listens for state updates.
- **Application Logic Layer (ViewModel**): Acts as the brain of the application (e.g. AuthViewModel, ResumeEnhancerViewModel). It processes user input, manages application state, and communicates with the Data Layer.
- **Data Access Layer (Repository):** Abstracts the data sources (e.g.,AuthRepository, ProfileRepository). It handles all network requests and data fetching strategies.
- **Domain Models:** Defines the structure of data objects (e.g., UserModel, StudentProfileModel) shared across all layers.

II. Cloud Services (Backend Infrastructure):

- **Firebase Authentication** [2]: Manages secure user login and signup.
- **Cloud Firestore** [2]**:** A NoSQL database storing user profiles and application data in collections (users, student_profiles, etc.).
- **Google Gemini** [3] **AI API:** The external AI service used for the "Smart Resume Enhancer" and "AI Chat" features.
- **Cloudinary** [4]**:** Cloud storage for hosting user profile images.

III. Data Flow:

- The flow is **unidirectional**: The UI triggers an action -> ViewModel processes it -> Repository fetches data -> External Service responds -> Repository returns a Model -> ViewModel updates the State -> UI rebuilds to show the new data.

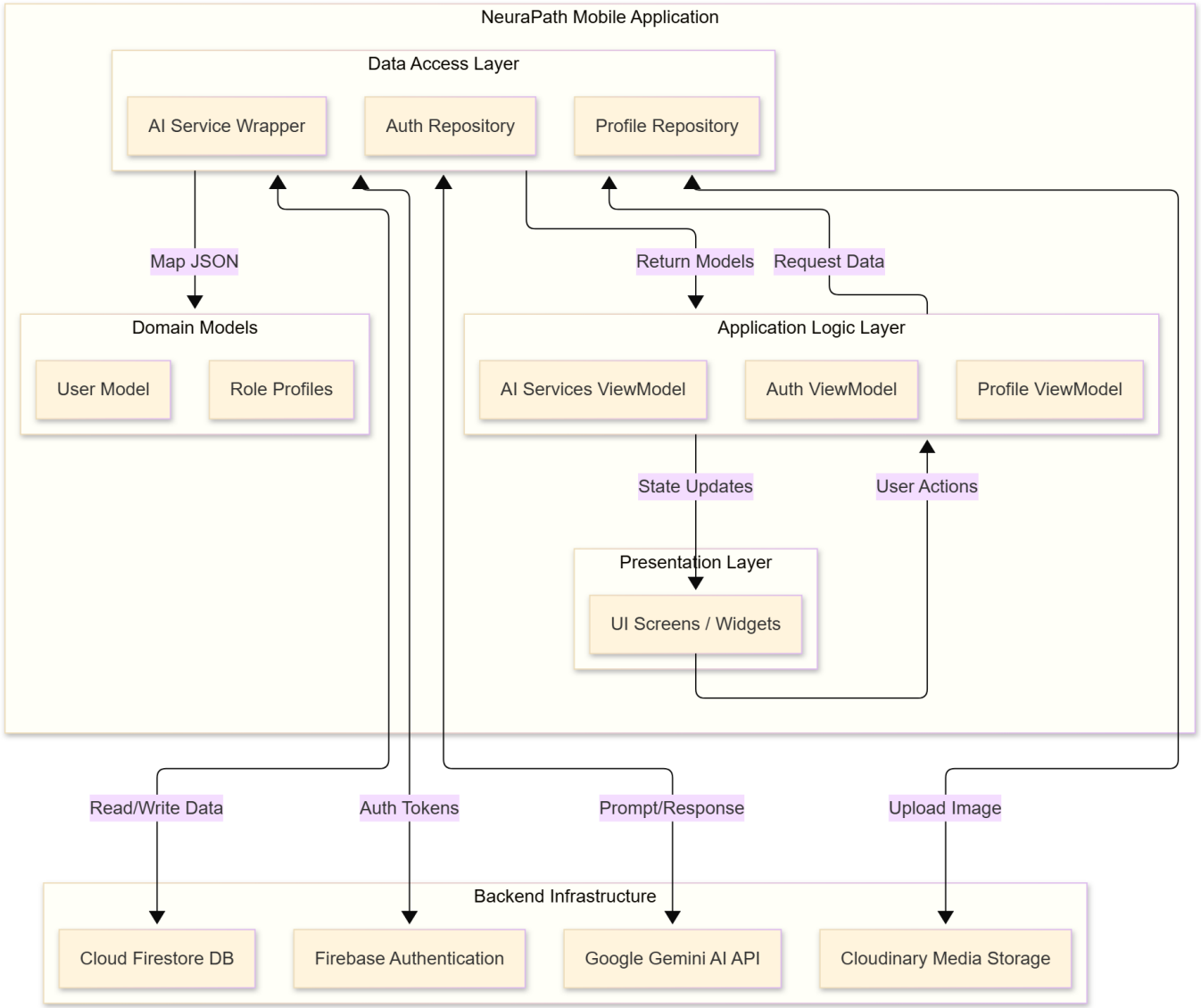


Figure 4.1: Architecture Design

5 Process Flow / Representation

The process flow of NeuraPath with representation is given below:

5.1 Major Process: Smart Resume Enhancement

The **Smart Resume Enhancer** is a core AI-driven feature of NeuraPath. It involves complex interactions between the local device (PDF processing) and the cloud (AI analysis).

5.1.1 Process Description

I. **Initiation:** The user selects a PDF resume from their device storage.
II. **Extraction:** The app uses a local PDF parsing engine to extract raw text from the document.
III. **Validation:** The system checks if the extracted text is sufficient for analysis (not empty, not scanned image).
IV. **AI Request:** The text is bundled with a specific "Persona Prompt" (instructing the AI to act as an HR Manager) and sent to the Gemini API [3].
V. **Analysis:** The AI evaluates the resume for ATS compatibility, formatting, and content quality.
VI. **Response Parsing:** The AI returns a structured JSON response containing a score (0-100), strengths, weaknesses, and keywords.
VII. **Presentation:** The app parses this JSON and renders a detailed report card to the user.

5.1.2 Activity Diagram

The Activity Diagram visualizes the sequential flow of control for the Resume Enhancement feature, highlighting the interaction between user actions, local processing, and remote API calls.

I. **Start**: The process initiates when the user triggers the "Upload Resume" action.
II. **File Selection & Extraction**: The system prompts for a file. Upon selection, the local PDF engine attempts to extract text.
III. **Validation (Decision Node):** The system checks if the extracted text is valid.
- Invalid: If empty or unreadable, it transitions to an Error State and prompts the user to retry.
- Valid: The process moves to prompt construction.

IV. **AI Interaction:** The system constructs a persona-based prompt and sends it to the Gemini API [3].
V. **API Response (Decision Node):** The system handles the network response.
- Failure: Network errors trigger the Error State.
- Success: A 200 OK response leads to JSON parsing.

VI. **Completion**: The parsed data is formatted into a UI report, and the process terminates at the **End Node**.

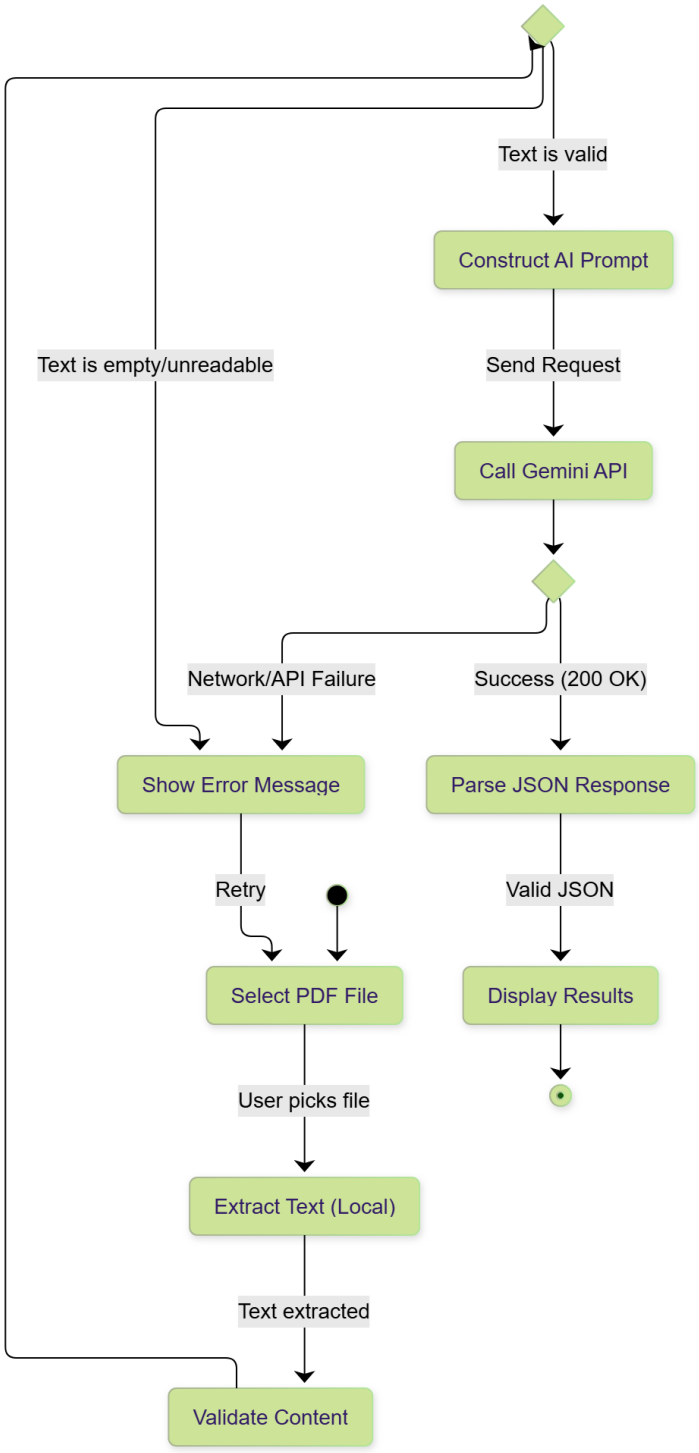


Figure 5.1: Activity Diagram

6 Design Models

All the four designed models with their descriptions and diagrammatic representations are given below:

6.1 Class Diagram

The Class Diagram illustrates the static structure of the system, showing the relationships between the core Data Models, ViewModels, and Repositories. It highlights the Object-Oriented principles of encapsulation and inheritance used in NeuraPath.

- **UserModel:** The base entity representing a registered user.
- **Role Profiles:** StudentProfileModel, JobSeekerProfileModel, and ProfessionalProfileModel contain role specific attributes.
- **AuthRepository:** Handles data persistence and authentication logic.
- **ViewModels:** AuthViewModel and ResumeEnhancerViewModel manage the application state and interact with the repository.

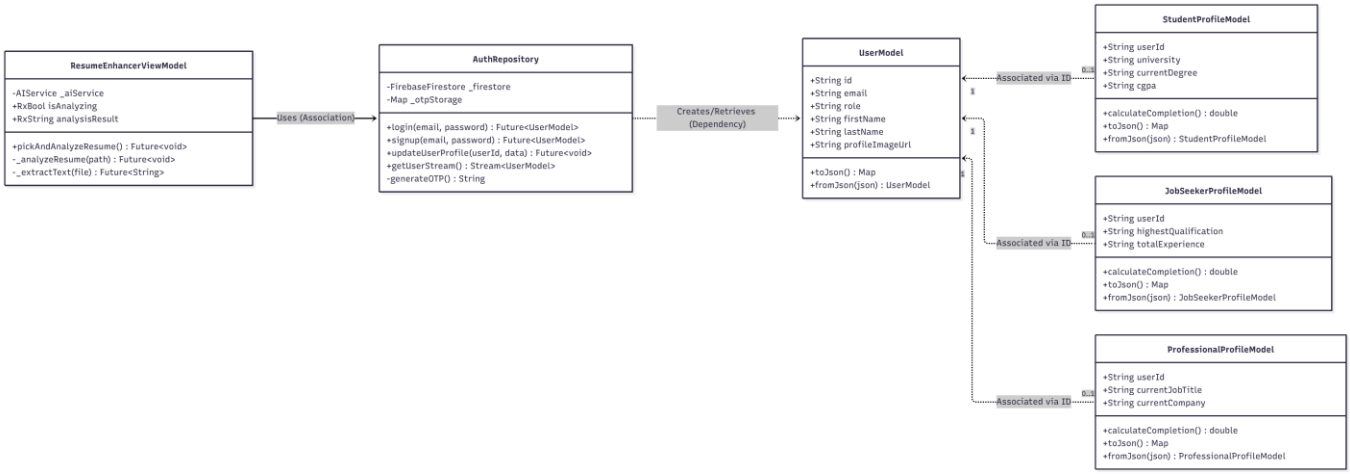


Figure 6.1: Class Diagram

6.2 Sequence Diagram

The Sequence Diagram details the dynamic behavior of the system during the Resume Analysis process. It shows the time-ordered interaction between the User, UI, ViewModel, and External Services.

I. **User Action:** The user picks a PDF file.
II. **Local Processing:** The ResumeEnhancerViewModel uses the PDFService (local library) to extract text.
III. **AI Request:** The extracted text is sent to the AIService.
IV. **External Call:** AIService calls the Google Gemini API [3].
V. **Response:** The API returns the analysis, which is displayed to the user.

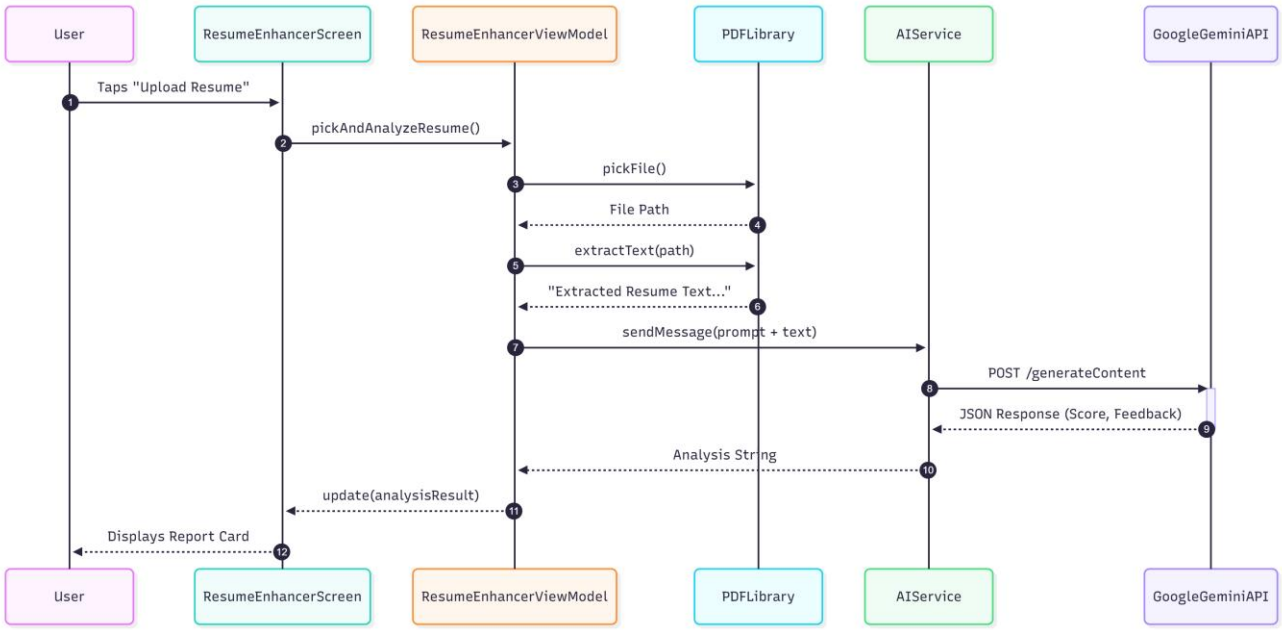


Figure 6.2: Sequence Diagram

6.3 State Transition Diagram

This diagram models the states of the Resume Enhancer feature, showing how the system transitions between states based on events.

- **Idle:** Waiting for user input.
- **Picking File:** User is selecting a document.
- **Analyzing:** System is processing the file (Loading state).
- **Success:** Results are displayed.
- **Error:** Something went wrong (e.g., invalid file).

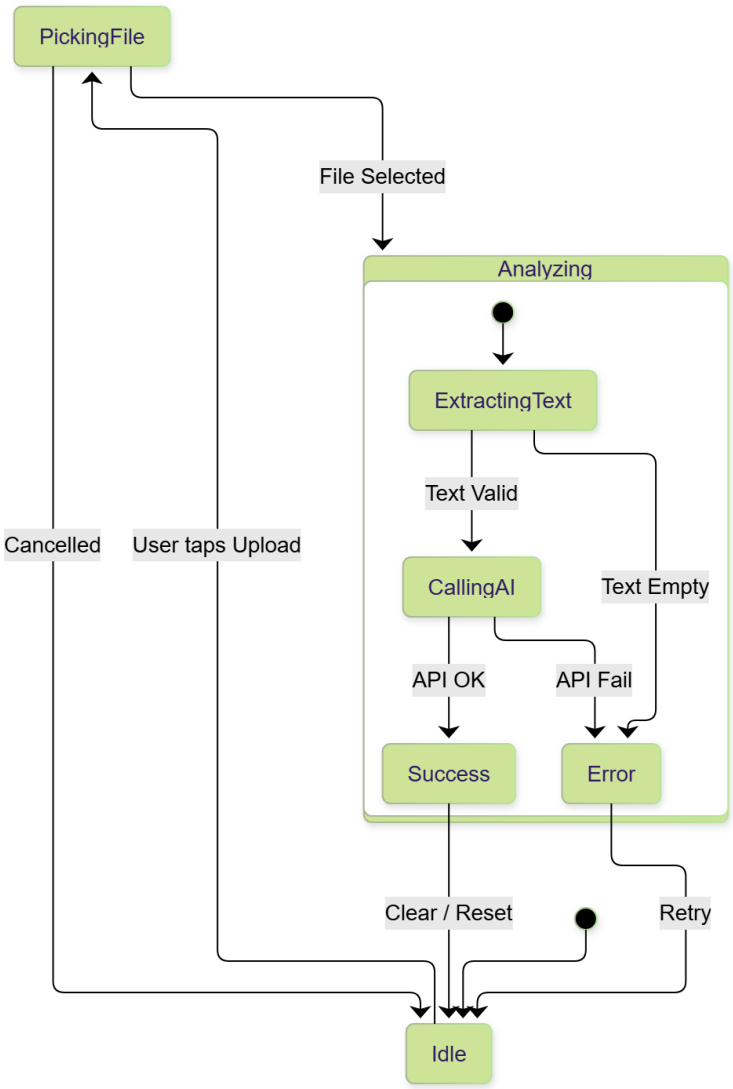


Figure 6.3: State Transition Diagram

6.4 Data Flow Diagrams (DFD)

The DFD Diagrams upto 3rd Level (0,1,2) with their descriptions and diagrams are given below:

6.4.1 DFD Level 0 (Context Diagram)

The Context Diagram represents the entire NeuraPath system as a single process interacting with external entities.

The Level 0 DFD, also known as the Context Diagram, provides a high-level view of the NeuraPath system. It treats the entire application as a single "Black Box" process.

- **External Entities:**
    - User: The primary actor who interacts with the system (Student, Job Seeker, or Professional).
    - Firebase Database: The external cloud database used for storing user credentials and profile data.
    - Gemini AI [3]: The external Artificial Intelligence service used for processing resumes and chat queries.

- **Data Flows:**
    - The User sends Credentials (Email/Password) and PDF Resumes to the System.
    - The System sends Auth Tokens, Career Advice, and Resume Feedback back to the User.
    - The System exchanges User Records with Firebase and sends Prompts to Gemini AI [3].

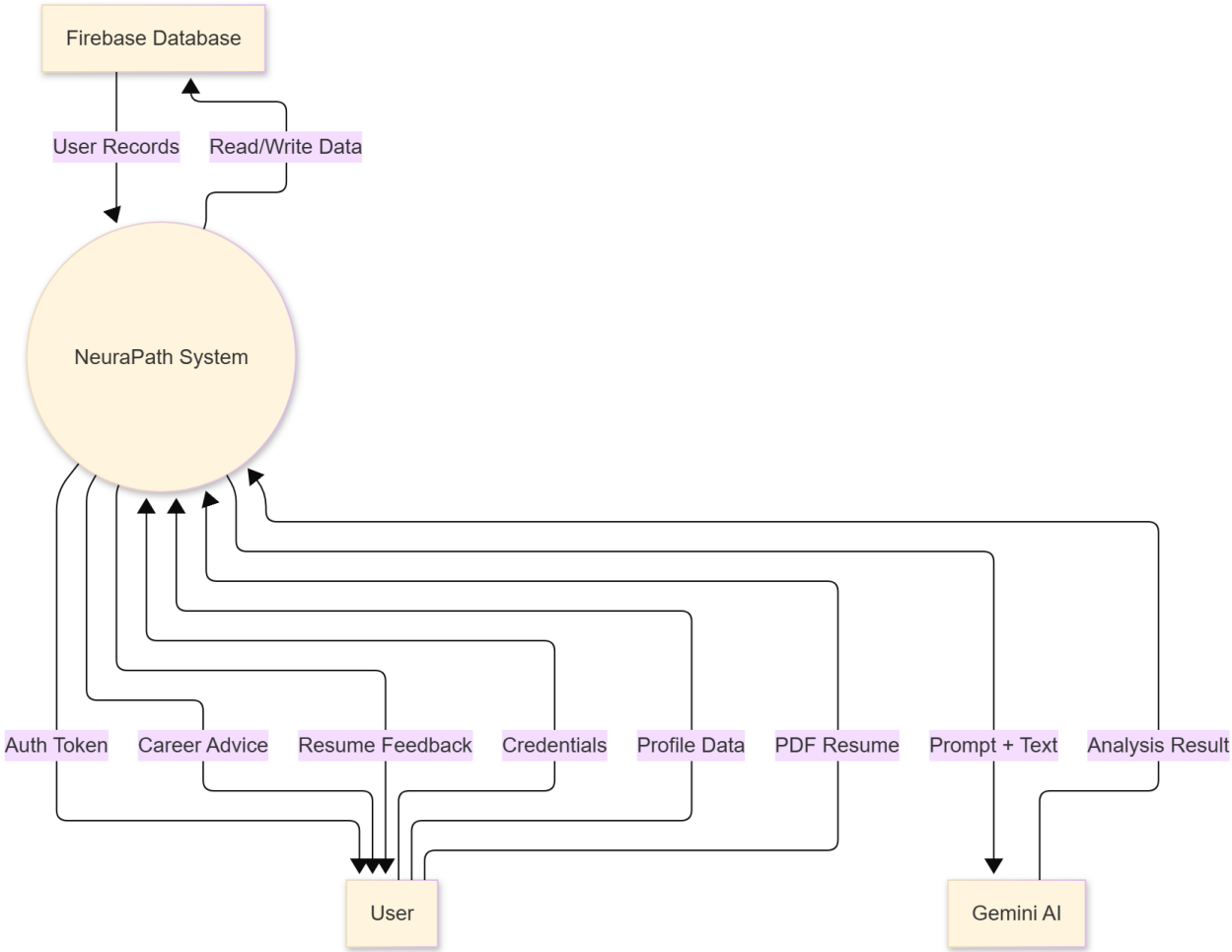


Figure 6.4: DFD Level 0 (Context Diagram)

6.4.2 DFD Level 1 (System Decomposition)

This level breaks down the main system into its primary functional modules.

The Level 1 DFD breaks down the single "NeuraPath System" process into its three major functional subsystems. This level reveals the internal data movement between these modules.

- **Processes:**
    - **1.0 Authentication:** Handles all login, signup, and session management tasks.
    - **2.0 Profile Management:** Manages the creation, reading, and updating of user profiles.
    - **3.0 AI Services:** Orchestrates the interaction with the Gemini AI [3] for resume analysis and chat.

- **Data Stores:**
    - **User DB:** Stores authentication details (Email, Password Hash).
    - **Profile DB:** Stores role-specific data (Education, Experience, Skills).

- **Flow:**
    - The User interacts with **1.0 Authentication** to gain access.
    - Once authenticated, the User can **Update Info** via **2.0 Profile Mgmt,** which saves data to the **Profile DB.**
    - The User can also **Upload Resume** to **3.0 AI Services,** which fetches context from the **Profile DB** before calling the **Gemini API** [3].

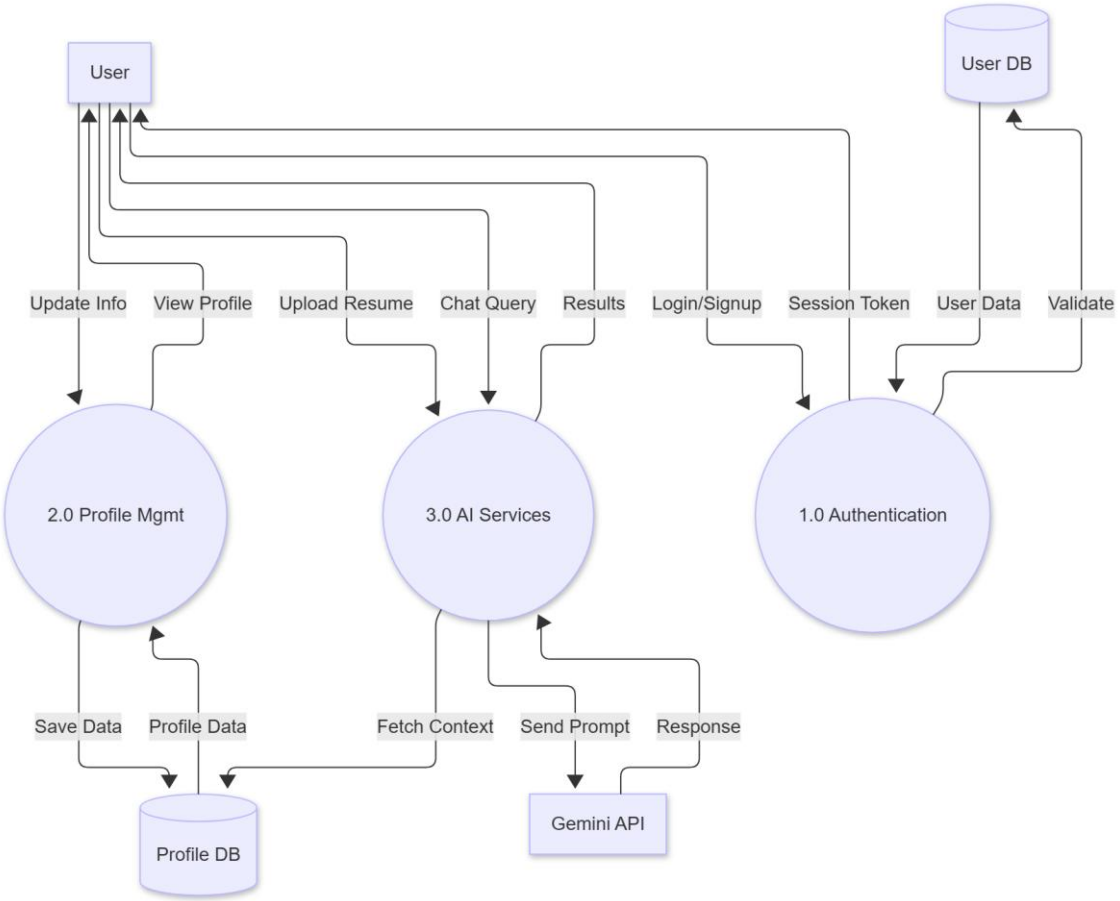


Figure 6.5: DFD Level 1 (System Decomposition)

6.4.3 DFD Level 2 (AI Services Expansion)

This level details the internal processes of the 3.0 AI Services module, specifically focusing on the Resume Enhancement feature.

The Level 2 DFD provides a granular view of the **3.0 AI Services** process, specifically focusing on the "Resume Enhancement" feature. It details the step-by-step transformation of data within this module.

- **Sub-Processes:**
    - **3.1 Upload & Parse:** Receives the PDF file and extracts raw text.
    - **3.2 Validate Content:** Checks if the text is readable and sufficient for analysis.
    - **3.3 Generate Prompt:** Combines the user's text with a pre-defined "Persona Prompt" for the AI.
    - **3.4 AI Analysis:** Sends the request to the external API and waits for the response.
    - **3.5 Format Results:** Parses the raw JSON response into a user-friendly format (Report Card).

- **Error Handling:** If **3.2 Validate Content** fails (e.g., empty PDF), an error is returned immediately to the User, bypassing the AI call.

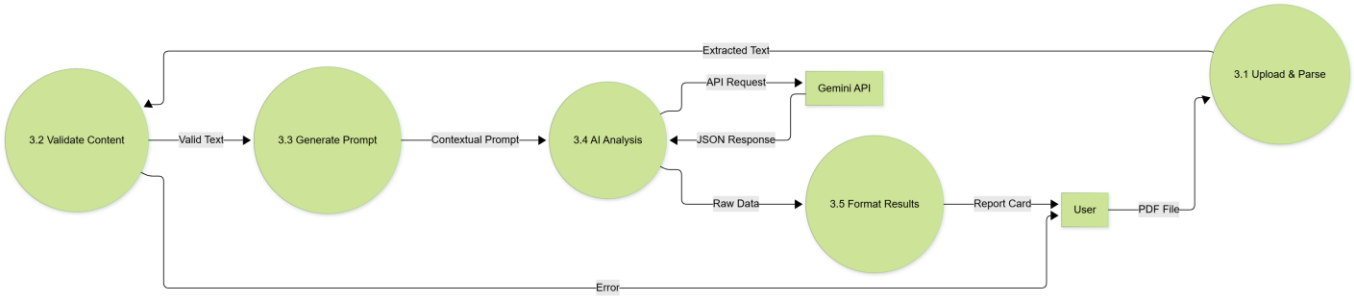


Figure 6.6: DFD Level 2 (AI Services Expansion)

7 Data Design

The Data Design of NeuraPath is explained below:

7.1 Information Domain Transformation

In NeuraPath, the information domain is transformed into data structures through a rigorous **Object-Relational Mapping (ORM)** style approach, adapted for a NoSQL environment.

I. **Input Phase:** User inputs (e.g., "CGPA: 3.8", "Skill: Flutter") are captured via Flutter UI Widgets (TextFormField).
II. **Object Instantiation:** These raw inputs are validated by the **ViewModel** and used to instantiate Dart Domain Models (e.g., **StudentProfileModel**). This ensures that data circulating within the app is strongly typed and consistent.
III. **Serialization:** Before storage, these Dart Objects are converted into **JSON Maps** (Map<String, dynamic>) using toJson() methods. This transformation is necessary for network transmission.
IV. **Persistence:** The JSON data is written to **Cloud Firestore** [2] documents. The hierarchical nature of JSON maps perfectly to Firestore's document-oriented structure.

7.2 Data Organization and Storage

The system entities are organized based on **Role-Based Partitioning** to ensure scalability and security.

7.2.1 Primary Storage: Cloud Firestore (NoSQL)

Data is stored in Collections (Folders) and Documents (Files).

- **“users” Collection**: Stores the "Core Identity" of every user. This is the central node linking to other data.
- **“student_profiles” Collection**: Stores academic data. The document ID matches the User ID, creating a 1:1 link.
- **“job_seeker_profiles” Collection:** Stores employment-readiness data.
- **“professional_profiles” Collection:** Stores career history data.

7.2.2 Secondary Storage: Cloudinary (Object Storage)

- **Profile Images:** Binary image data is not stored in the database. Instead, images are uploaded to Cloudinary [4], and the resulting **Public URL** is stored in the users collection (profileImageUrl).

7.2.3 Local Storage: GetStorage

- **Session Persistence:** Lightweight user preferences and session tokens are stored locally on the device to allow the user to stay logged in between app restarts.

8 Data Dictionary

The following is an alphabetical list of the major System Objects (Classes), their attributes, and methods, reflecting the Object-Oriented design of NeuraPath.

8.1 AuthRepository

Handles all interactions with Firebase Authentication and the "**users**" collection in Firestore [2].

Table 8.1: AuthRepository

Object Member	Type	Description
Methods		
login(email, password)	Future<UserModel>	Authenticates user and retrieves profile.
signup(email, password)	Future<UserModel>	Creates new auth account and user document.
generateAndSendOTP(email)	Future<String>	Generates 4-digit code and triggers email service.
verifyOTP(email, enteredOTP)	Future<bool>	Validates the user-provided code against storage.
updateUserProfile(...)	Future<void>	Updates core fields (Name, Phone, Role, Image).
isProfileComplete(userId)	Future<bool>	Checks if mandatory fields are filled.

8.2 JobSeekerProfileModel

Represents the extended profile data for a user with the "Job Seeker" role.

Table 8.2: JobSeekerProfileModel

Object Member	Type	Description
Attributes		
userId	String	Foreign Key linking to UserModel.
highestQualification	String	e.g., "Bachelors", "Masters".
totalExperience	String	e.g., "2 Years".
expectedSalaryRange	String	e.g., "50k - 80k".
resumeSummary	String	Short professional bio.
technicalSkills	String	Comma-separated list of hard skills.
atsScore	String	Last calculated resume score.
Methods		
toJson()	Map<String, dynamic>	Serializes object for Firestore.
fromJson(json)	JobSeekerProfileModel	Factory constructor for deserialization.
calculateCompletion()	double	Returns percentage (0-100) of filled fields.

8.3 ProfessionalProfileModel

Represents the extended profile data for a user with the "Professional" role.

Table 8.3: ProfessionalProfileModel

Object Member	Type	Description
Attributes		
userId	String	Foreign Key linking to UserModel.
currentJobTitle	String	e.g., "Senior Software Engineer".
currentCompany	String	Name of current employer.
yearsOfExperience	String	Total professional tenure.
skills	String	Professional skillset.
desiredNextJobRole	String	Career growth target.
Methods		
toJson()	Map<String, dynamic>	Serializes object for Firestore.
fromJson(json)	ProfessionalProfileModel	Factory constructor for deserialization.
calculateCompletion()	double	Returns percentage (0-100) of filled fields.

8.4 ResumeEnhancerViewModel

Manages the state and logic for the AI Resume Enhancer feature.

Table 8.4: ResumeEnhancerViewModel

Object Member	Type	Description
Attributes		
isAnalyzing	RxBool	Observable state for loading indicator.
analysisResult	RxString	Stores the AI's feedback text.
selectedFilePath	RxString	Path to the user's PDF file.
Methods		
pickAndAnalyzeResume()	Future<void>	Orchestrates file picking and analysis flow.
_analyzeResume(path)	Future<void>	Private method to extract text and call AI service.
clearAnalysis()	void	Resets the view state.

8.5 StudentProfileModel

Represents the extended profile data for a user with the "Student" role.

Table 8.5: StudentProfileModel

Object Member	Type	Description
Attributes		
userId	String	Foreign Key linking to UserModel.
university	String	Name of academic institution.
currentDegree	String	e.g., "BS Computer Science".
cgpa	String	Academic performance metric.
major	String	Field of study.
expectedGraduationYear	String	e.g., "2025".
preferredIndustryDomains	String	e.g., "FinTech, HealthTech".
Methods		
toJson()	Map<String, dynamic>	Serializes object for Firestore.
fromJson(json)	StudentProfileModel	Factory constructor for deserialization.
calculateCompletion()	double	Returns percentage (0-100) of filled fields.

8.6 UserModel

The core user entity containing authentication and identification details.

Table 8.6: UserModel

Object Member	Type	Description
Attributes		
id	String	Unique User ID (Primary Key).
email	String	Registered email address.
role	String	'Student', 'Job Seeker', or 'Professional'.
firstName	String	User's first name.
lastName	String	User's last name.
profileImageUrl	String	URL to Cloudinary image.
Methods		
toJson()	Map<String, dynamic>	Serializes object for Firestore.
fromJson(json)	UserModel	Factory constructor for deserialization.

9 Algorithm & Implementation

This section provides a systematic look at the core algorithms implemented within the system's major components. Since NeuraPath follows an Object-Oriented Design, the algorithms are presented as member functions of their respective classes using Pseudo Code (PDL).

9.1 Class: AuthRepository

The **AuthRepository** class serves as the central gateway for all authentication-related operations and user data persistence. It encapsulates direct interactions with **Firebase Authentication** [2] and **Cloud Firestore** [2], providing a clean API for the rest of the application to manage user sessions and profile data securely.

9.1.1 Function: login(email, password)

Authenticates a user against the Firestore database and retrieves their profile.

```
FUNCTION login(email, password)
// Step 1: Query Firestore for user with matching email
QUERY result = Firestore.collection('users').where('email', EQUALS email).get()
// Step 2: Check if user exists
IF result.isEmpty THEN
THROW Exception("User not found")
END IF
// Step 3: Verify Password
UserDocument doc = result.first
IF doc.data['password'] NOT EQUALS password THEN
THROW Exception("Incorrect password")
END IF
// Step 4: Create Session
SET _currentUserId = doc.id
// Step 5: Return User Model
RETURN UserModel.fromJson(doc.data)
END FUNCTION
```

9.1.2 Function: signup(email, password)

Registers a new user and creates their initial record in the database.

```
FUNCTION signup(email, password)
// Step 1: Check for existing user
QUERY existing = Firestore.collection('users').where('email', EQUALS
email).get()
IF existing.isNotEmpty THEN
THROW Exception("User already exists")
END IF
// Step 2: Create new User Document
DocumentReference ref = Firestore.collection('users').doc()
// Step 3: Construct User Object
UserModel newUser = NEW UserModel(
```

```
id: ref.id,
email: email,
password: password
)
// Step 4: Save to Database
AWAIT ref.set(newUser.toJson())
// Step 5: Set Session
SET _currentUserId = ref.id
RETURN newUser
END FUNCTION
```

9.2 Class: ResumeEnhancerViewModel

The ResumeEnhancerViewModel acts as the intermediary between the UI and the AI services, managing the state of the resume analysis process. It handles file selection, local text extraction, and orchestrates the asynchronous communication with the Gemini [5] API to deliver actionable feedback to the user.

9.2.1 Function: pickAndAnalyzeResume()

Orchestrates the process of selecting a file, extracting text, and requesting AI analysis.

```
FUNCTION pickAndAnalyzeResume()
TRY
// Step 1: Open File Picker
File file = AWAIT FilePicker.pickFiles(type: PDF)
IF file IS NULL THEN
RETURN // User cancelled
END IF
// Step 2: Update State
SET selectedFilePath = file.path
SET isAnalyzing = TRUE
// Step 3: Extract Text (Local Processing)
String text = AWAIT PDFService.extractText(file.path)
IF text IS EMPTY THEN
THROW Exception("Cannot read PDF")
END IF
// Step 4: Prepare AI Prompt
String prompt = "Analyze this resume for ATS score, strengths, and weaknesses: "
+ text
// Step 5: Call AI Service
String result = AWAIT AIService.sendMessage(prompt)
// Step 6: Update UI
SET analysisResult = result
CATCH Exception e
SHOW Snackbar("Error", e.message)
FINALLY
SET isAnalyzing = FALSE
END TRY
END FUNCTION
```

9.3 Class: StudentProfileModel

The StudentProfileModel is a data transfer object that encapsulates all academic and skill-related information specific to student users. It includes logic for serialization to JSON for database storage and a utility method to calculate the profile's completeness percentage based on filled fields.

9.3.1 Function: calculateCompletion()

Calculates the profile completion percentage based on filled fields.

```
FUNCTION calculateCompletion()
// Step 1: Define Total Tracked Fields
INTEGER totalFields = 21
INTEGER filledFields = 0
// Step 2: Iterate and Check Fields
IF currentDegree IS NOT EMPTY THEN INCREMENT filledFields
IF major IS NOT EMPTY THEN INCREMENT filledFields
IF university IS NOT EMPTY THEN INCREMENT filledFields
IF cgpa IS NOT EMPTY THEN INCREMENT filledFields
// ... (repeat for all fields) ...
IF technicalSkills IS NOT EMPTY THEN INCREMENT filledFields
// Step 3: Calculate Weighted Percentage
// Base 15% is for basic info (Name, Email) which is mandatory
DOUBLE percentage = 15.0 + (85.0 * (filledFields / totalFields))
RETURN percentage
END FUNCTION
```

10 Software Requirements Traceability Matrix

The following table summarizes how each software requirement has been met by specific design components and functions.

Table 10.1: Software Requirements Traceability Matrix

Req. Number	Ref. Item	Design Component	Component Items (Functions)
FR-01	User Login	**Class Diagram** (AuthRepository)	login(email, password)
FR-02	User Signup	**Class Diagram** (AuthRepository)	signup(email, password)
FR-03	OTP Verification	**Class Diagram** (AuthRepository)	generateAndSendOTP(email), verifyOTP(email, code)
FR-04	Role Selection	**DFD Level 1** (Process 1.0)	signup() (Role assignment logic)
FR-05	Edit Profile	**Class Diagram** (AuthRepository)	updateUserProfile(userId, data)
FR-06	View Student Profile	**Class Diagram** (StudentProfileViewModel)	loadProfile(userId)
FR-07	Calculate Profile Progress	**Class Diagram** (StudentProfileModel)	calculateCompletion()
FR-08	Upload Resume PDF	**DFD Level 2** (Process 3.1)	pickAndAnalyzeResume()
FR-09	Extract PDF Text	**Sequence Diagram** (PDFLibrary)	extractText(path)
FR-10	AI Resume Analysis	**Sequence Diagram** (AIService)	sendMessage(prompt)
FR-11	Display Analysis Results	**State Diagram** (Success State)	update(analysisResult)
FR-12	AI Chat Interaction	**DFD Level 1** (Process 3.0)	sendMessage(prompt)
FR-13	Logout	**Class Diagram** (AuthRepository)	logout()

11 Human Interface Design

The Human Interface Design of NeuraPath is explained below with screen images:

11.1 User Perspective Functionality

NeuraPath is designed with a "User-First" philosophy, ensuring that complex AI features are accessible through intuitive interfaces.

11.1.1 Onboarding & Authentication:

- Upon launching the app, the user is presented with a clean **Login Screen.** New users can navigate to **Sign Up**, where they perform a critical action: selecting their role (Student, Job Seeker, Professional). This single decision configures the entire subsequent app experience.
- **Feedback**: Real-time validation on email/password fields (e.g., "Password must be 6+ chars") and loading spinners during network requests.

11.1.2 Dashboard (Home Screen):

- Post-login, the user lands on a **Role-Specific Dashboard**.
  - Students see academic tools.
  - Job Seekers see resume tools.
- The layout uses a **Grid System** to display feature cards (e.g., "Smart Resume Enhancer", "AI Chat").
- Feedback: Tapping a card provides immediate visual feedback (ripple effect) and navigates to the feature.

11.1.3 AI Resume Enhancer:

- The user taps "Upload Resume" and selects a PDF. The app displays a "Analyzing..." state with a progress indicator.
- Once complete, a **Report Card** appears, showing the ATS Score in a large, color-coded circle (Green for high, Red for low), followed by scrollable lists of strengths and weaknesses.

11.1.4 Profile Management:

- Users can view their digital identity in the **Profile Tab**. A progress bar at the top visually indicates how "complete" their profile is, encouraging them to add more details.
- The **Edit Profile** screen allows for quick updates to contact info without overwhelming the user with too many fields at once.

11.2 Screen Images

The images of our NeuraPath Mobile App (real screens) are given below:

11.2.1 Launcher Screen (Splash)

A sleek, branded entry screen featuring the NeuraPath logo centered on a clean background. It briefly displays the app version before automatically transitioning to the Login screen.

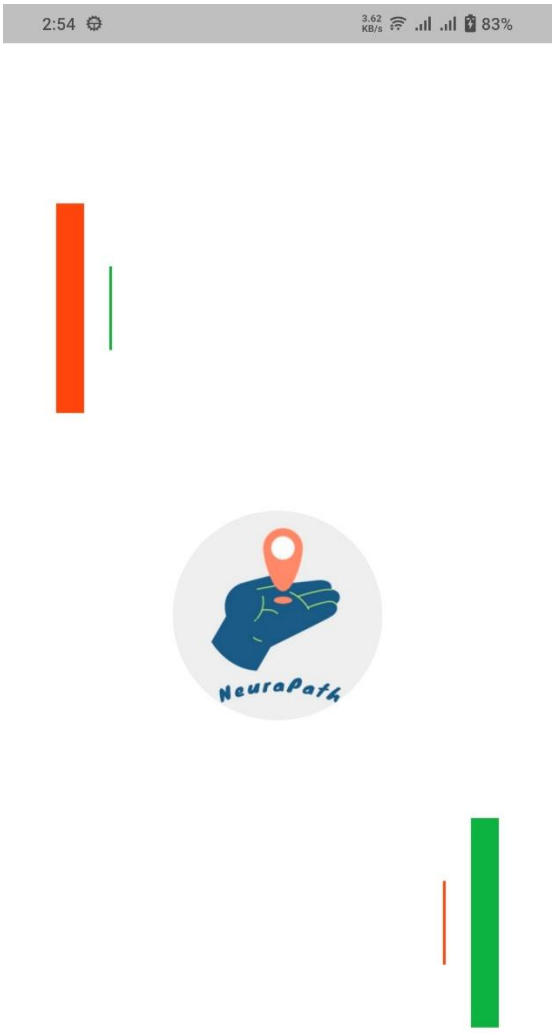


Figure 11.1: Launcher Screen (Splash)

11.2.2 Login Screen

A minimal screen with the NeuraPath logo at the top. Two text fields for Email and Password. A primary "Login" button in Sage Green (#8BA989) and a secondary text link for "Forgot Password?".

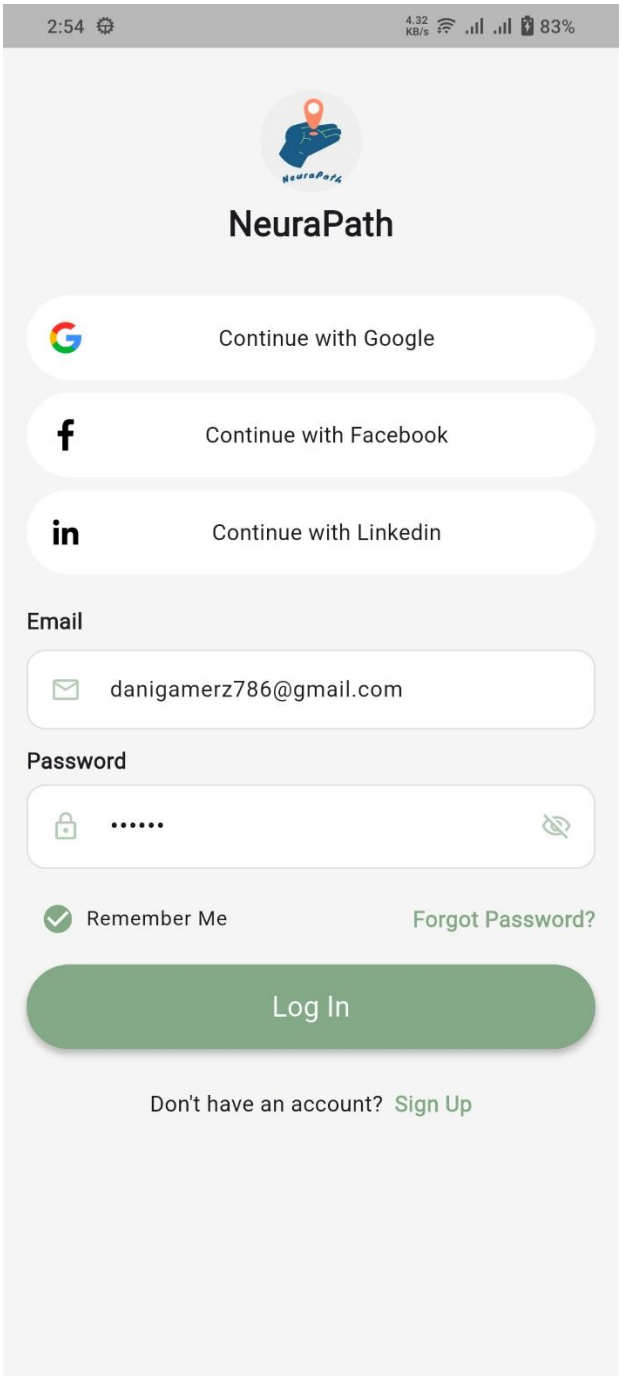


Figure 11.2: Login Screen

11.2.3 Student Home Screen

A dashboard showing a welcome message "Hello, [Name]". A grid layout containing cards for "Smart Resume Enhancer" (with a document icon) and "AI Job Matcher". A curved bottom navigation bar is visible.

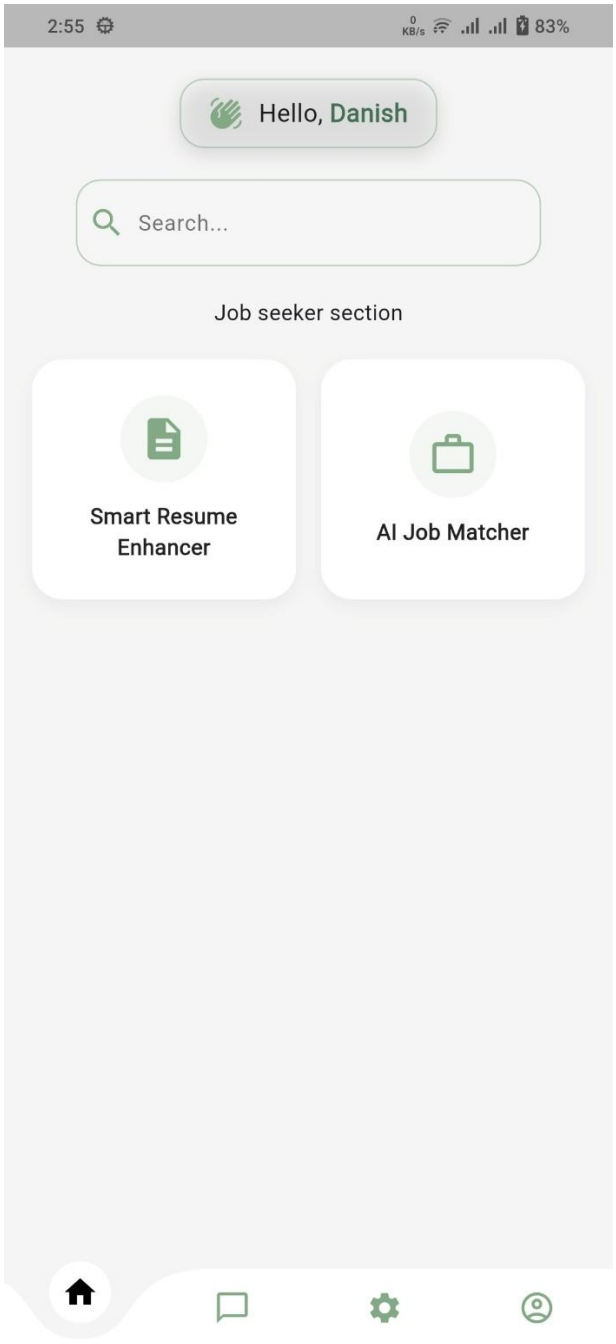


Figure 11.3: Student Home Screen

11.2.4 Smart Resume Analysis

Displays a large circular gauge showing the ATS Score (e.g., 75/100). Below it, expandable sections for "Strengths", "Weaknesses", and "Keywords".

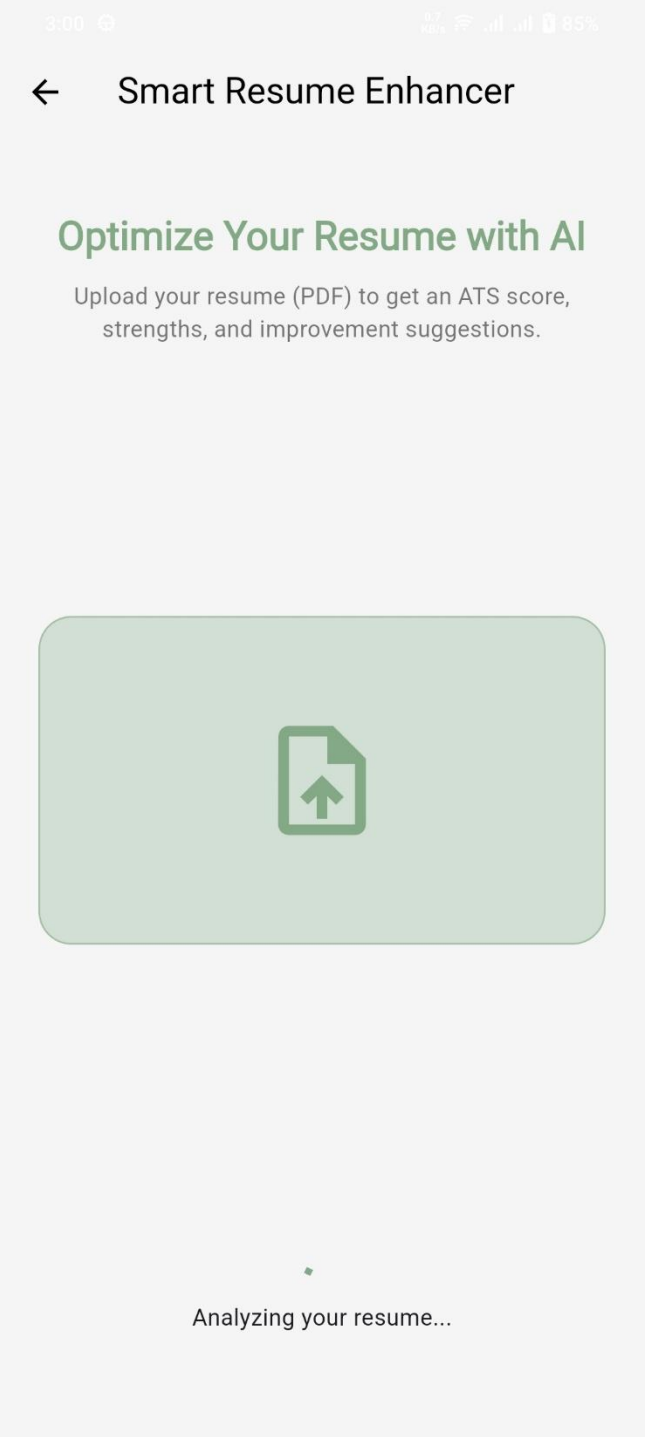


Figure 11.4: Resume Analysis

11.2.5 AI Chat Screen

A conversational interface similar to standard messaging apps. User messages are right-aligned (Sage Green bubbles), and AI responses are left-aligned (Grey bubbles). A text input field and send button are fixed at the bottom.

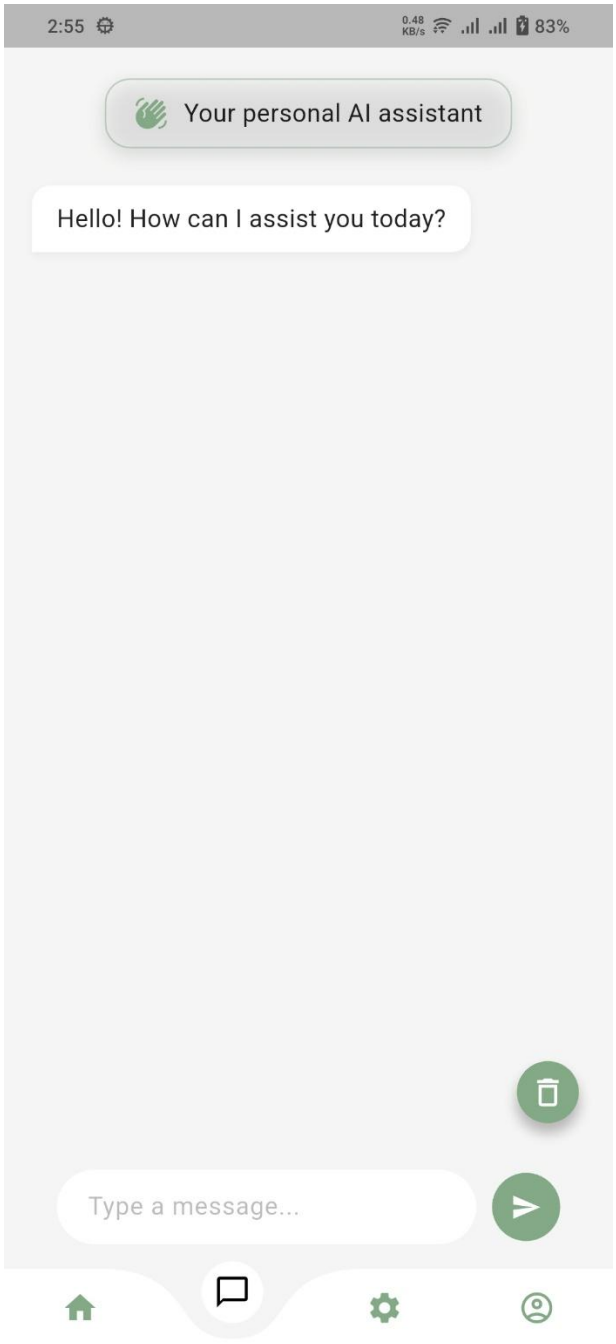


Figure 11.5: AI Chat Screen

11.2.6 Settings Screen

A list of configuration options including "Edit Profile", "Change Password", "Privacy Policy", and "Logout". Each item is represented by a leading icon and a trailing arrow for navigation.

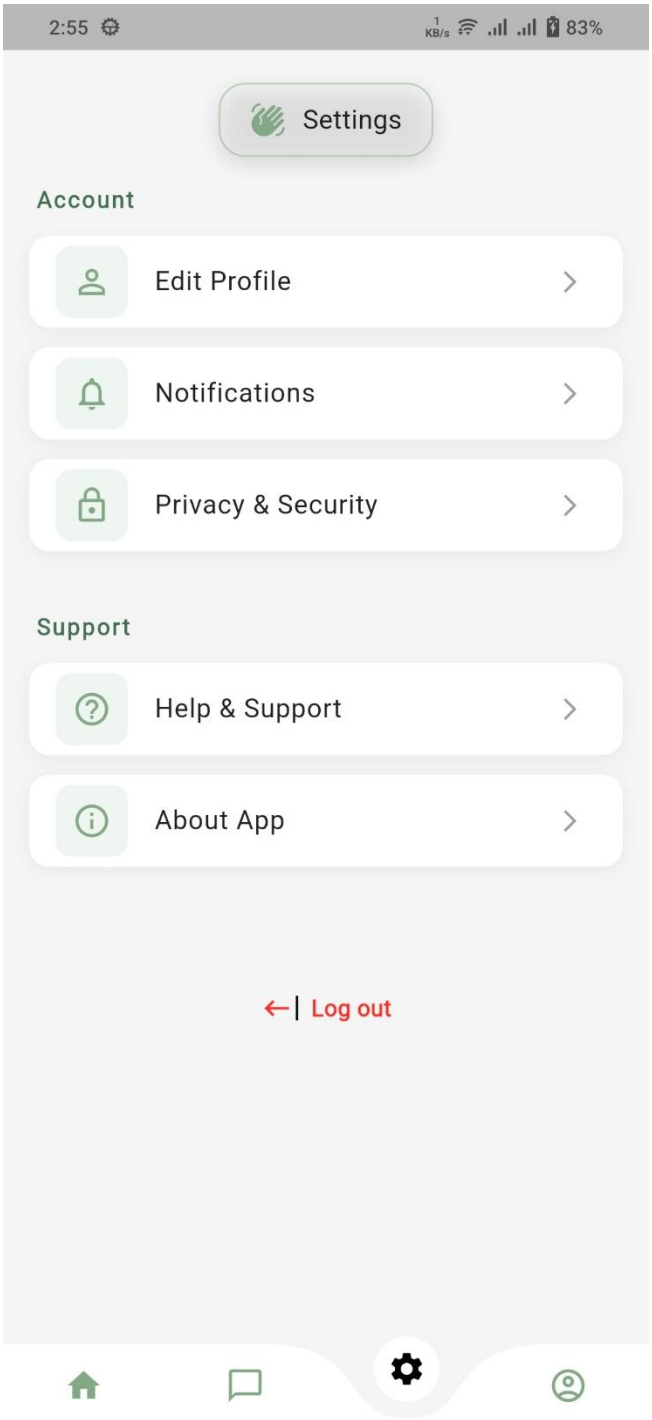


Figure 11.6: Settings Screen

11.2.7 Profile Management Screen

A detailed view of the user's professional identity. It features a profile header with the user's image and role. Below, a tabbed or scrollable view displays sections for "Education", "Skills", and "Experience", with "Edit" icons for each section.

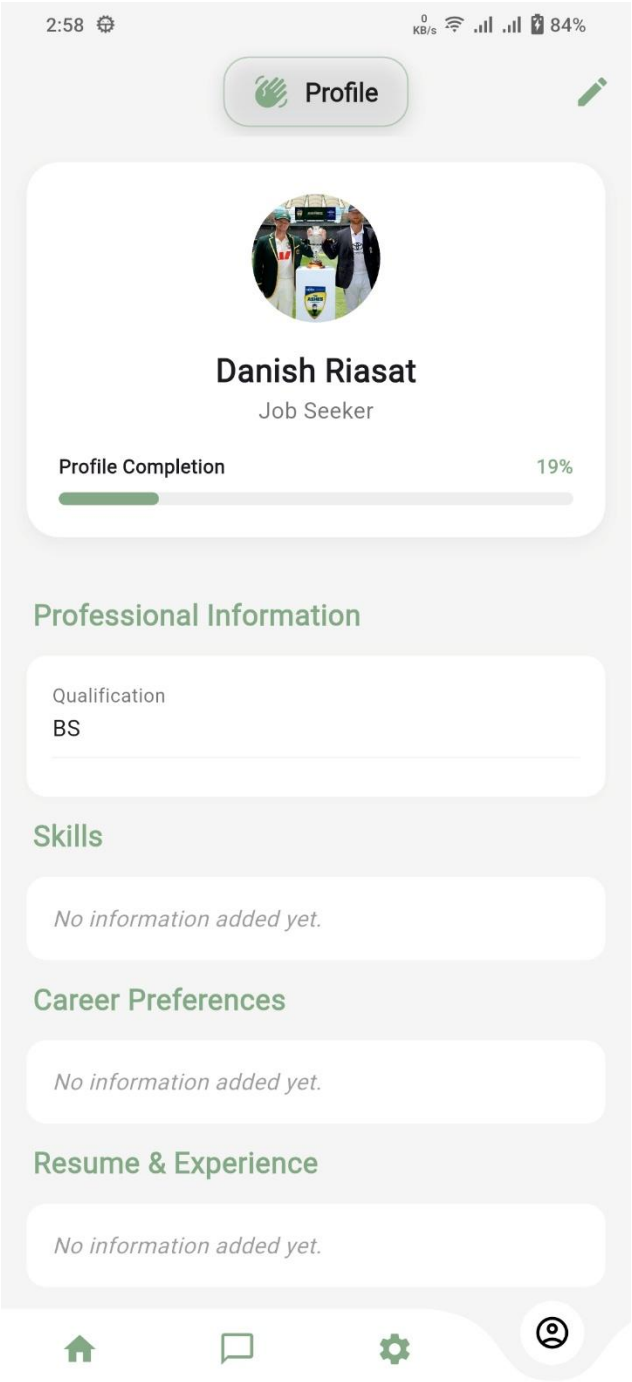


Figure 11.7: Profile Management Screen

11.3 Screen Objects and Actions

The following table details the interactive elements found on major screens and their associated system actions.

Table 11.1: Screen Objects and Actions

Screen Name	Object (UI Element)	Action (User Interaction)	System Response
Login Screen	EmailField	Type text	Validates email format.
	LoginButton	Tap	Calls AuthRepository.login(). Shows loading spinner. Navigates to Home on success.
	SignupLink	Tap	Navigates to SignupScreen.
Signup Screen	RoleDropdown	Select Item	Sets the role variable. Updates UI to show role-specific hints.
	SignupButton	Tap	Calls AuthRepository.signup(). Creates user in Firestore.
Home Screen	ResumeCard	Tap	Navigates to ResumeEnhancerScreen.
	BottomNavBar	Tap Icon	Switches between Home, Chat, Settings, and Profile tabs.
Resume Screen	UploadButton	Tap	Opens System File Picker (PDF only).
	AnalyzeButton	Tap	Triggers ResumeEnhancerViewModel.pickAndAnalyzeResume(). Sends file to AI.
Profile Screen	EditFAB (Icon)	Tap	Navigates to EditProfileScreen.
	ProgressBar	View	Visualizes calculateCompletion() result.
Settings Screen	LogoutTile	Tap	Shows Confirmation Dialog. If "Yes", clears session and routes to Login.
AI Chat Screen	MessageInput	Type text	Updates input buffer.
	SendButton	Tap	Sends message to Gemini [5] API. Appends user message to chat list immediately.

12 Appendices

The Appendices are given below:

12.1 Appendix A: Gemini AI Persona Prompt

The following prompt is used to configure the Gemini [5] Generative AI model for the **Smart Resume Enhancer** feature. This ensures the AI adopts the persona of an experienced HR Manager.

"Act as a Senior HR Manager and ATS Specialist with 15 years of experience. I will provide you with the raw text extracted from a candidate's resume. Your task is to:

I. Analyze the resume for ATS (Applicant Tracking System) compatibility.
II. Identify key strengths and selling points.
III. Identify critical weaknesses, formatting errors, or missing sections.
IV. Suggest 5-10 high-impact industry keywords missing from the profile.
V. Assign a quantitative score out of 100 based on modern hiring standards.

Return the output in the following strict JSON format: { 'score': 0-100, 'strengths': ['point 1', 'point 2'], 'weaknesses': ['point 1', 'point 2'], 'keywords': ['keyword 1', 'keyword 2'] }"

12.2 Appendix B: Third-Party Libraries

The following key open-source packages were utilized in the development of NeuraPath:

- **get** [5]**:** State management and route management.
- **firebase_auth** [2]**:** Backend authentication services.
- **cloud_firestore:** NoSQL cloud database.
- **google_generative_ai:** Dart SDK for accessing Gemini API.
- **read_pdf_text:** Local PDF text extraction engine.
- **file_picker:** Native file explorer integration.

13 References

[1] Google Developers, 2025. [Online]. Available: https://flutter.dev/docs.

[2] C. Google, 2025. [Online]. Available: https://firebase.google.com/docs.

[3] "Google AI. (2024).," [Online]. Available: https://ai.google.dev/docs.

[4] Cloudinary Ltd., 2025. [Online]. Available: https://cloudinary.com/documentation.

[5] "Jonsson,J. (2023).," [Online]. Available: https://pub.dev/packages/get.